

# CS-405: Deep Learning

## Large Language Models

Dr. Sajjad Hussain

SEECS, NUST

April 28th, 2026

# Large Language Models (LLMs)

## Definition

Large Language Models are neural network-based models trained on **massive amounts of text data** to understand and generate human-like text.

## What makes them "LARGE"?

### Model Size

- Billions to trillions of parameters
- GPT-3: 175B
- GPT-4: approx 1.7T

### Training Data

- Hundreds of billions to trillions of tokens
- Internet, books, code
- Multi-lingual

### Compute

- Thousands of GPUs/TPUs
- Weeks to months of training
- Millions of dollars

**LLMs = Scale + Data + Compute = Emergent Abilities!**

# Types of LLMs: By Architecture

## Three Main Architectural Families

Based on the Transformer building blocks.

| Type            | Architecture   | Examples                | Best For                      |
|-----------------|----------------|-------------------------|-------------------------------|
| Encoder-Only    | Bidirectional  | BERT, RoBERTa, ALBERT   | Understanding, Classification |
| Decoder-Only    | Autoregressive | GPT-3/4, LLaMA, Mistral | Generation, Chat, Code        |
| Encoder-Decoder | Full Seq2Seq   | T5, BART, Pegasus       | Translation, Summarization    |

### Quick Comparison:

- **Encoder-Only:** “Read and understand” (masked language modeling)
- **Decoder-Only:** “Read and continue” (causal language modeling)
- **Encoder-Decoder:** “Read and transform” (sequence-to-sequence)

Architecture choice depends on the task!

# Types of LLMs: By Access and Size

## By Access Model

| Type            | Access         | Examples               | Pros/Cons                 |
|-----------------|----------------|------------------------|---------------------------|
| Open Source     | Weights public | LLaMA, Mistral, Falcon | Customizable, transparent |
| Open Access     | API only       | GPT-4, Claude, Gemini  | Powerful, black box       |
| Research Access | Application    | PaLM, Early GPT-3      | Cutting edge, limited     |

## By Size Category

| Category   | Parameters | Examples                 |
|------------|------------|--------------------------|
| Small      | < 1B       | DistilBERT, TinyLLaMA    |
| Medium     | 1B – 10B   | BERT-large, GPT-2 medium |
| Large      | 10B – 100B | LLaMA 2 70B, GPT-3       |
| Very Large | 100B – 1T  | GPT-4, Gemini Ultra      |

Bigger doesn't always mean better for every task!

# Key Terminology: Pre-training

## What is Pre-training?

The initial phase where the model learns language from **massive, unlabeled text data** by predicting masked tokens or next tokens.

## How it works (Masked Language Modeling for BERT):

Input: "The [MASK] sat on the [MASK]"

Target: "cat" and "mat"

Loss: Cross-entropy on masked positions

## What the model learns:

- Vocabulary and syntax
- Grammar and word order
- World knowledge and facts
- Reasoning patterns
- Cultural context

**Pre-training = Learning language from scratch!**

# Key Terminology: Fine-tuning

## What is Fine-tuning?

Further training of a pre-trained model on **task-specific labeled data**.

## Types of Fine-tuning:

| Type             | Description                  | Data Size                      |
|------------------|------------------------------|--------------------------------|
| Full Fine-tuning | Update all parameters        | 10K-100K examples              |
| Instruction FT   | Learn to follow instructions | 10K-1M (instruction, response) |
| PEFT (LoRA)      | Train small adapters only    | 100-10K examples               |
| RLHF             | Learn from human preferences | 10K-100K comparisons           |

## Parameter-Efficient Fine-Tuning (PEFT):

- Only 0.1% to 1% of parameters are trained!
- LoRA adds low-rank matrices to attention layers
- Saves memory and time

**Fine-tuning adapts general models to specific tasks!**

# Key Terminology: Zero-shot and Few-shot Learning

## Zero-shot Learning

The model performs a task **without seeing any examples**.

Input: "Classify as positive or negative: 'Great product!' "  
Output: "positive"

## Few-shot Learning (In-Context Learning)

The model learns from **a few examples provided in the prompt**.

Examples: "Great!" → Positive, "Terrible!" → Negative  
Input: "Average" → ?  
Output: "Neutral"

## Why this matters:

- No gradient updates needed!
- Leverages pre-trained knowledge

# Retrieval-Augmented Generation (RAG)

## What is RAG?

A technique that **retrieves relevant documents** from an external knowledge source and **augments the prompt** before generation.

## The RAG Pipeline:

- 1 **Retrieve:** Search vector database for relevant documents
- 2 **Augment:** Add retrieved documents to the prompt
- 3 **Generate:** LLM produces answer based on context

## Why RAG?

- **Up-to-date information:** No retraining needed
- **Reduces hallucination:** Answers grounded in retrieved text
- **Citations:** Can show sources for verification
- **Private data:** Access internal documents securely

User: "What did the CEO say in yesterday's meeting?"

RAG: Retrieves transcript → Augments prompt → Generates answer with quotes



# LLM Challenges: Hallucination

## What is Hallucination?

The model generates **confident but factually incorrect information**.

Q: "Who won the Nobel Prize in Physics in 1875?"

A: "Albert Einstein won the Nobel Prize in Physics in 1875."

(Fact: No prize awarded in 1875. Einstein won in 1921.)

## Why it happens:

- LLMs are pattern matchers, not knowledge bases
- No true understanding of truth or falsehood
- Training data contains contradictions
- Model prioritizes fluency over accuracy

## Mitigations:

- Retrieval-Augmented Generation (RAG)
- Self-consistency checks
- Citation generation
- Reinforcement learning with factuality rewards

# LLM Challenges: Bias and Fairness

## What is Bias in LLMs?

Systematic, unfair preferences or prejudices embedded in model outputs that disadvantage certain groups.

## Types of Bias:

| Bias Type          | Example                                          |
|--------------------|--------------------------------------------------|
| Gender Bias        | “CEO” → “he”, “nurse” → “she”                    |
| Racial/Ethnic Bias | Certain names associated with crime or poverty   |
| Cultural Bias      | Western-centric worldviews (Christmas vs Diwali) |
| Socioeconomic Bias | Non-standard English penalized                   |
| Political Bias     | Leaning toward certain ideologies                |

## Bias vs Fairness:

- **Bias** = The problem (measurable patterns)
- **Fairness** = The goal (value-based, subjective)

# LLM Challenges: Safety, Toxicity, Privacy

## Safety & Toxicity

Models can generate harmful, offensive, or dangerous content.

### Examples:

- Hate speech and harassment
- Instructions for illegal activities
- Self-harm promotion

**Mitigations:** Safety fine-tuning (RLHF), content filtering, usage policies

## Privacy Concerns

- **Memorization:** Models can reproduce training data (including PII)
- **Prompt leakage:** Users accidentally sharing sensitive data
- **Model inversion:** Attacks can extract training examples

**Mitigations:** Differential privacy, data deduplication, PII redaction

# LLM Challenges: Computational Cost and More

## Computational Cost

| Model       | Parameters  | Training Cost (est.) |
|-------------|-------------|----------------------|
| GPT-3       | 175B        | \$4-12 million       |
| LLaMA 2 70B | 70B         | \$2-5 million        |
| GPT-4       | approx 1.7T | \$100-200 million    |

**Environmental Impact:** Training can emit hundreds of tons of CO<sub>2</sub>

## Other Challenges

- **Catastrophic Forgetting:** Fine-tuning on new tasks causes forgetting old capabilities
- **Lack of True Understanding:** Statistical pattern matching, not genuine reasoning
- **Limited Context Window:** Cannot process extremely long documents

# Open-access LLM Ecosystem: Hugging Face

## What is Hugging Face?

The “[GitHub for Machine Learning](#)” — a platform for sharing models, datasets, and applications.

## Core Components:

| Component    | Description                      | Scale             |
|--------------|----------------------------------|-------------------|
| Model Hub    | Repository of pre-trained models | 500,000+ models   |
| Datasets     | Repository of datasets           | 100,000+ datasets |
| Spaces       | Hosting for ML demos             | Interactive apps  |
| Transformers | Library for loading/using models | PyTorch/TF        |
| PEFT         | Parameter-efficient fine-tuning  | LoRA, AdaLoRA     |
| Accelerate   | Distributed training             | Multi-GPU         |

## Popular Open Models on Hub:

- LLaMA 2/3 (Meta), Mistral, Mixtral, Gemma (Google)
- Zephyr, Phi-2, Falcon, BLOOM

# Hugging Face: How to Use

## Simple Pipeline Usage

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
result = classifier("I love this product!")
# Output:  [{'label': 'POSITIVE', 'score': 0.999}]
```

## Loading Any Model

```
from transformers import AutoModelForCausalLM,
AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b")
tokenizer = AutoTokenizer.from_pretrained("
meta-llama/Llama-2-7b")
```

# Hugging Face: Parameter-Efficient Fine-tuning (LoRA)

## LoRA Fine-tuning Example

```
from peft import LoraConfig, get_peft_model

lora_config = LoraConfig(
    r=8, lora_alpha=32,
    target_modules=["q_proj", "v_proj"]
)
peft_model = get_peft_model(base_model, lora_config)

# Only trains approx 0.1% of parameters!
```

## Benefits of LoRA:

- Drastically reduces memory usage
- Faster training
- Easy to swap adapters for different tasks
- No inference latency overhead

# Model Licenses: Important Consideration

## Not All Open Models Are Equal!

Licenses determine how you can use the model.

| License           | Commercial Use?           | Examples                 |
|-------------------|---------------------------|--------------------------|
| MIT/Apache 2      | Yes                       | BERT, T5, GPT-2, Mistral |
| Llama 2/3 License | Yes (restrictions apply)  | LLaMA 2, LLaMA 3         |
| Llama 1 License   | No (research only)        | LLaMA 1                  |
| OpenAI License    | No (weights not released) | GPT-3, GPT-4             |

### Before Using a Model:

- Check the license on the model card
- Verify commercial usage rights
- Review attribution requirements
- Check for usage restrictions (e.g., > 700M users requires Meta approval)

**Always check the license before deploying!**



# Summary: LLMs Key Takeaways

- ➊ **Definition:** Large neural networks trained on massive text data
- ➋ **Architectures:** Encoder-only (BERT), Decoder-only (GPT), Encoder-Decoder (T5)
- ➌ **Training:** Pre-training (from scratch) → Fine-tuning (task-specific)
- ➍ **Key Capabilities:** Zero-shot, Few-shot, In-context learning
- ➎ **RAG:** Retrieval + Generation = Grounded, up-to-date answers
- ➏ **Challenges:** Hallucination, Bias, Safety, Privacy, Cost
- ➐ **Ecosystem:** Hugging Face provides open models, tools, and community

LLMs are powerful but require responsible deployment!